



Data Analysis Bootcamp



eCommerce Analysis from Raw Data to Business Decisions

By : Team 17
Team Leader : Eslam Ashraf



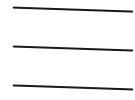


Table of contents

01

Introduction

02

**Data Architecture &
Source Integration**

03

**Data Cleaning &
Transformation**

04

**Exploratory Data
Analysis (EDA) in
SSMS**

05

**Advanced Data
Analysis in Power
BI**

06

**Dashboard
Walkthrough**

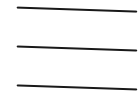


Table of contents

07

**Insights &
Recommendations**

08

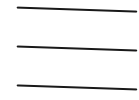
**The Extra Mile: AI-
Augmented Analysis**

09

Conclusion

10

Q&A



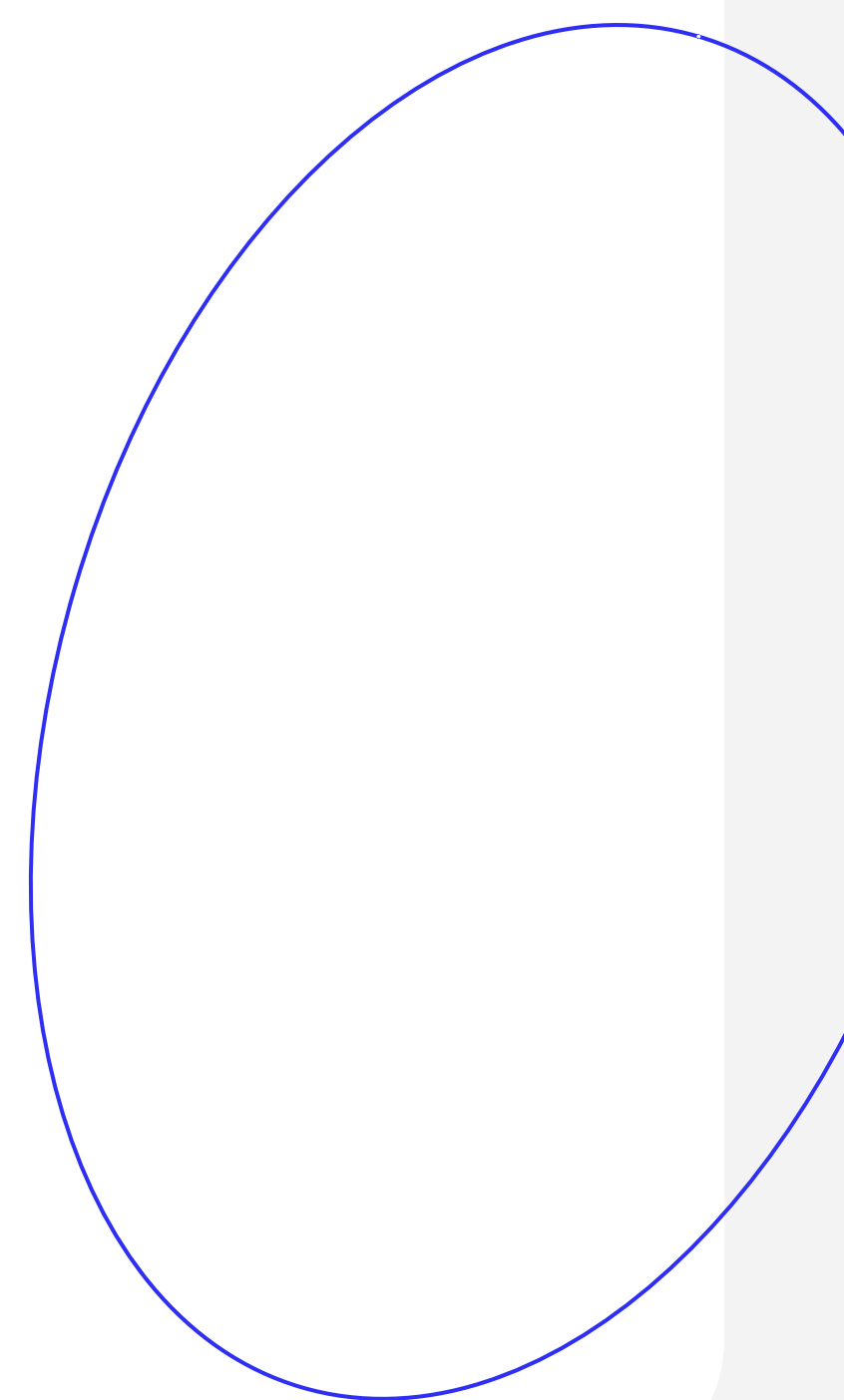
Introduction

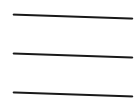
Objective

Provide data-driven insights to optimize sales, understand customer behavior, and drive eCommerce growth

Team Collaboration

Tools used: Notion for task tracking, documentation, and real-time collaboration.
o Roles distribution and project timeline.



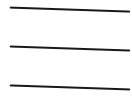


★ Active Timeline Board **All** +

≡ ⬇ ⚡ 🔍 ↗ ... **New** ▾

🎯 Projects

Aa Project name	⚙ Status	👤 Owner	📅 Dates	⬇ Priority	📊 Completion	+ ...
📅 PHASE 1: Kickoff & Planning	● Done	E Eslam Ashraf	May 10, 2025 → May 11, 2025	High	100.00%	
🔍 PHASE 2: Data Source Identification & Access	● Done	E Eslam Ashraf	May 11, 2025 → May 12, 2025	High	100.00%	
☁ PHASE 3: Azure Environment Setup	● In progress	A Abdelrahman Saa	May 20, 2025 → May 21, 2025	Low	0.00%	
🔒 PHASE 4: Security & Compliance for Azure	● In progress	A Abdelrahman Saa	May 20, 2025 → May 21, 2025	Low	0.00%	
🗄 PHASE 5: Data Modeling in SQL Server	● Done	E Eslam Ashraf 🌙 im vengeance H Heba Adel Y Yava Hm	May 11, 2025 → May 16, 2025	High	100.00%	



PHASE 4: Security & Compliance for Azure 4

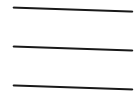
PHASE 5: Data Modeling in SQL Server 5

PHASE 6: Data Cleaning & Transformation 3

PHASE 7: Analysis & KPI Development 12

Aa Task name	⚙ Status	👤 Assignee	📅 Due	⬇ Priority	☰ Summary
Calculate the total sales revenue from all orders	Done	im vengeance	May 16, 2025	High	
Compute the 30-day customer retention rate after their first purchase.	Done	im vengeance	May 16, 2025	High	
List the top 5 best-selling products by quantity sold.	Done	im vengeance	May 16, 2025	High	
Identify customers with the highest number of orders.	Done	Eslam Ashraf	May 16, 2025	High	
Recommend products frequently bought together with items in customer wishlists.	Done	Eslam Ashraf	May 16, 2025	High	





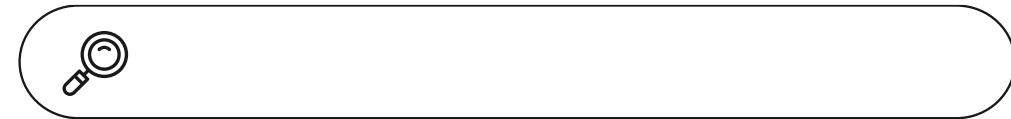
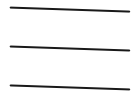
Data Architecture & Source Integration

Data Source

Connected structured datasets stored securely on Microsoft Azure SQL Database.

Why Azure

Scalability, security, and real-time access for all collaborators.



[Home](#) >

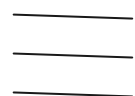
Create SQL Database ...

Microsoft

Subscription	Azure for Students
Resource group	Abdelrahman-Saad
Region	UAE North
Database name	Abdelrahman-Ecommerce
Server	(new) abdelrahman-server
Authentication method	SQL and Microsoft Entra authentication
Server admin login	Abdelrahman-Saad
Microsoft Entra Admin	42110425.Abdelrahman@acu.edu.eg
Compute + storage	General Purpose - Serverless: Standard-series (Gen5), 1 vCore, 32 GB storage, zone redundant disabled
Backup storage redundancy	Locally-redundant backup storage

Networking

Allow Azure services and resources to access this server	Yes
Add current client IP address 197.57.132.169	Yes
Private endpoint	None
Minimum TLS version	1.2
Connection Policy	Default



Home > Microsoft.SqlDatabase.newDatabaseNewServer_51641a319cdf475a9ff02 | Overview >



Abdelrahman-Ecommerce (abdelrahman-server/Abdelrahman-Ecommerce)

SQL database

Search

Copy Restore Export Set server firewall Delete Connect with... Feedback

Overview

Activity log

Tags

Diagnose and solve problems

Query editor (preview)

Mirror database in Fabric (preview)

Resource visualizer

> Settings

> Data management

> Integrations

> Power Platform

> Security

> Intelligent performance

> Monitoring

> Automation

> Help

Mirror databases in Microsoft Fabric Easily replicate your existing databases in Fabric, and help your team achieve streamlined ETL and operational analytics goals. [Learn more](#)

Essentials

Resource group (move) : [Abdelrahman-Saad](#)

Status : Online

Location : UAE North

Subscription (move) : [Azure for Students](#)

Subscription ID : d8b652df-3a4c-4181-a8bc-71a8d9810184

Tags (edit) : [Add tags](#)

Server name : [abdelrahman-server.database.windows.net](#)

Connection strings : [Show database connection strings](#)

Pricing tier : [General Purpose - Serverless: Gen5, 1 vCore](#)

Auto-pause delay : [1 hour](#)

Earliest restore point : No restore point available

[JSON View](#)

Getting started

Monitoring

Properties

Features

Notifications (0)

Integrations

Tutorials

Start working with your database

Connect to your database and start working with data with a few simple steps. [Learn more](#)



Configure access

Configure network access to your SQL



Connect to application

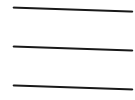
Use connection strings to connect to your SQL



Start developing

Work in your database by using tools to add,

Add or remove favorites by pressing Ctrl+Shift+F



Connect to Server

SQL Server

Login

Connection Properties

Always Encrypted

Additional Connection Parameters

Server

Server type: Database Engine

Server name: abdelrahman-server

Authentication: SQL Server Authentication

Login: Abdelrahman-Saad

Password: *****

☒ Remember password

Connection Security

Encryption: Mandatory

☐ Trust server certificate

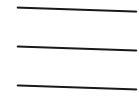
Host name in certificate:

Connect

Cancel

Help

Options <<



Data Cleaning & Transformation

Tools Used

SQL Server Management Studio (SSMS)

Steps Performed

- Null & duplicate checks, date/format standardization.
- Joining tables, managing discounts, handling outliers.
- Stored procedures and window functions for performance.



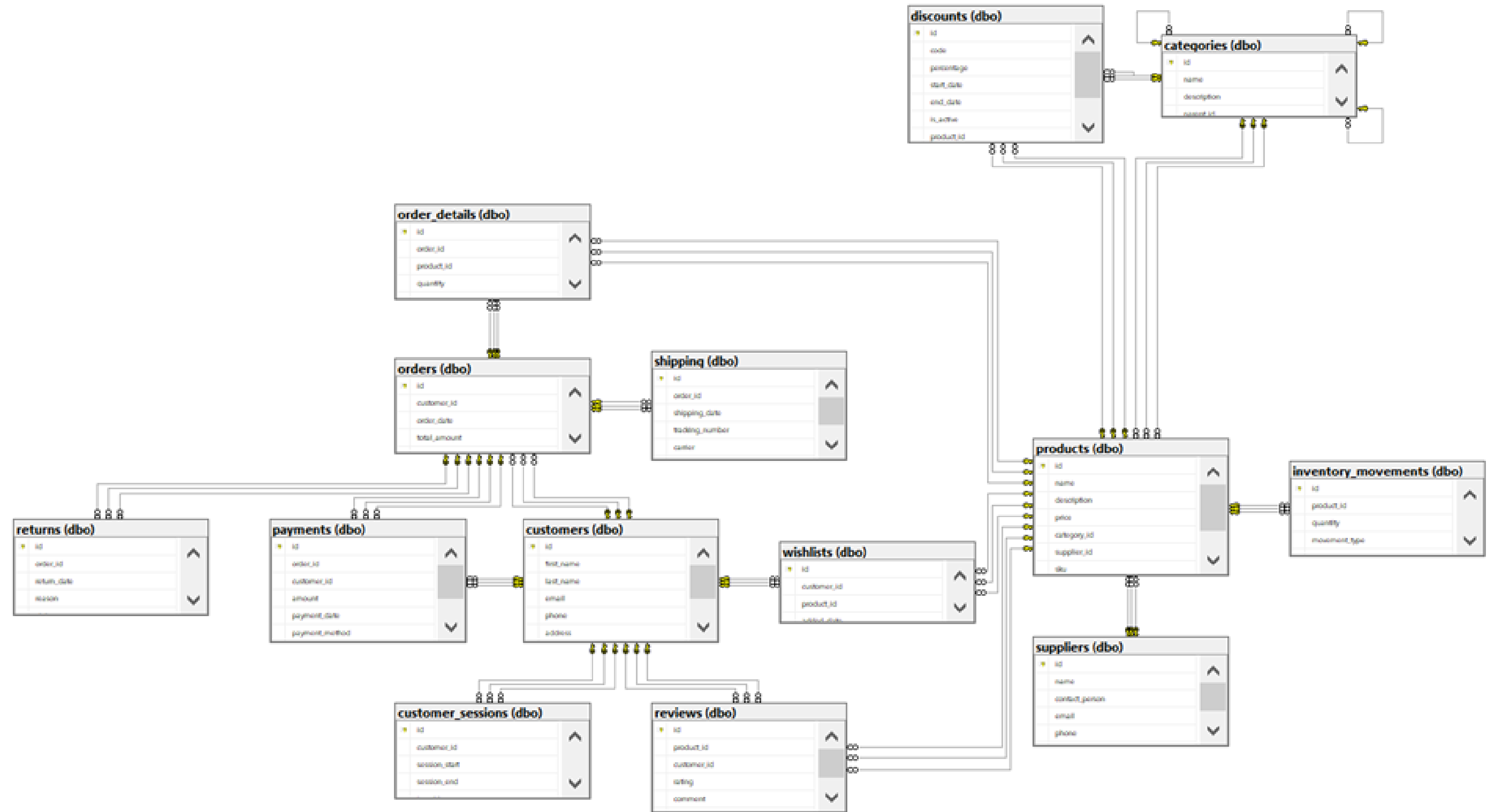
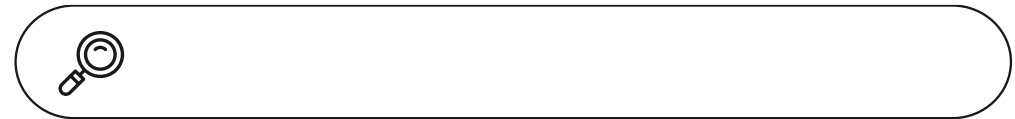
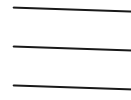
Database Dictionary

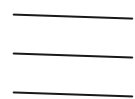
Table: categories

id	INT	Primary key. Unique identifier for each category.
name	VARCHAR	Name of the category (e.g., 'Part').
description	TEXT	Description or details about the category.
parent_id	INT	Refers to another category id (self-referencing for nested categories).

Table: customer_sessions

id	INT	Primary key. Unique identifier for each session.
customer_id	INT	Foreign key referencing customers(id).
session_start	DATETIME	Session start timestamp.
session_end	DATETIME	Session end timestamp.
ip_address	VARCHAR	IP address used in the session.





Cleaning the data : Checking the duplicates

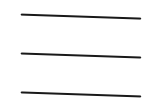
-- check for duplicates

```
SELECT id, COUNT(*) FROM categories GROUP BY id HAVING COUNT(*) > 1;
SELECT id, COUNT(*) FROM customers GROUP BY id HAVING COUNT(*) > 1;
SELECT id, COUNT(*) FROM customer_sessions GROUP BY id HAVING COUNT(*) > 1;
SELECT id, COUNT(*) FROM discounts GROUP BY id HAVING COUNT(*) > 1;
SELECT id, COUNT(*) FROM inventory_movements GROUP BY id HAVING COUNT(*) > 1;
SELECT id, COUNT(*) FROM order_details GROUP BY id HAVING COUNT(*) > 1;
SELECT id, COUNT(*) FROM orders GROUP BY id HAVING COUNT(*) > 1;
SELECT id, COUNT(*) FROM payments GROUP BY id HAVING COUNT(*) > 1;
SELECT id, COUNT(*) FROM products GROUP BY id HAVING COUNT(*) > 1;
SELECT id, COUNT(*) FROM returns GROUP BY id HAVING COUNT(*) > 1;
SELECT id, COUNT(*) FROM reviews GROUP BY id HAVING COUNT(*) > 1;
SELECT id, COUNT(*) FROM shipping GROUP BY id HAVING COUNT(*) > 1;
SELECT id, COUNT(*) FROM suppliers GROUP BY id HAVING COUNT(*) > 1;
SELECT id, COUNT(*) FROM wishlists GROUP BY id HAVING COUNT(*) > 1;
```

91 %

Results Messages

id	(No column name)



Checking for nulls

```
-- check for nulls
SELECT * FROM customers
WHERE first_name IS NULL
   OR last_name IS NULL
   OR email IS NULL
   OR phone IS NULL
   OR registration_date IS NULL;

SELECT * FROM orders
WHERE customer_id IS NULL
   OR order_date IS NULL
   OR total_amount IS NULL
   OR status IS NULL;

SELECT * FROM categories
WHERE id IS NULL
   OR name IS NULL
   OR description IS NULL
   OR parent_id IS NULL;
```

83 %

Results Messages

id	first_name	last_name	email	phone	address	registration_date
----	------------	-----------	-------	-------	---------	-------------------

id	customer_id	order_date	total_amount	status
----	-------------	------------	--------------	--------

	id	name	description	parent_id
1	1	Part	Win color election her. Idea society understand ar...	NULL
2	2	Program	Remain great line dog alone either level. Arm mo...	NULL
3	3	Large	Table people job civil here. View seat according st...	NULL
4	4	Hold	Yard call test investment state hundred. Scene h...	NULL
5	5	Speech	Forward upon provide. Live world bring if continue...	NULL
6	1...	Table	Although him again natural yeah maintain. Call up...	NULL



Dynamic SQL to check all tables in a database

Instead of repeating the same query for each table, we used a dynamic SQL query that scans all tables in the database and reports the number of NULLs per column.

The script uses the INFORMATION_SCHEMA.COLUMNS view to automatically build and run the NULL-check query.

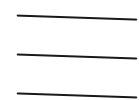
```
-- Dynamic SQL to check all tables in a database

DECLARE @sql NVARCHAR(MAX) = '';

SELECT @sql = @sql +
    'SELECT ''' + TABLE_NAME + ''' AS table_name, ''' + COLUMN_NAME + ''' AS column_name, COUNT(*) AS null_count ' +
    'FROM [' + TABLE_NAME + '] WHERE [' + COLUMN_NAME + '] IS NULL UNION ALL '
FROM INFORMATION_SCHEMA.COLUMNS
WHERE TABLE_SCHEMA = 'dbo';
SET @sql = LEFT(@sql, LEN(@sql) - 10); -- Remove last UNION ALL

EXEC sp_executesql @sql;
```

.00 %			
Results Messages			
	table_name	column_name	null_count
1	categories	description	0
2	categories	id	0
3	categories	name	0
4	categories	parent_id	15
5	customer_sessions	customer_id	0
6	customer_sessions	id	0
7	customer_sessions	ip_address	0
8	customer_sessions	session_end	0
9	customer_sessions	session_start	0
10	customers	address	0



Checking for outliers

```
- SELECT
    id,
    name,
    price
FROM products
WHERE price > (
    SELECT AVG(price) + (3 * STDEV(price))
    FROM products
)
OR price < (
    SELECT AVG(price) - (3 * STDEV(price))
    FROM products
);
```

100 %

Results Messages

id	name	price
----	------	-------



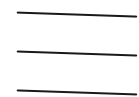
Orders Table

```
SELECT
    id,
    customer_id,
    total_amount
FROM orders
WHERE total_amount > (
    SELECT AVG(total_amount) + (3 * STDEV(total_amount))
    FROM orders
)
OR total_amount < (
    SELECT AVG(total_amount) - (3 * STDEV(total_amount))
    FROM orders
);
```

100 % ▾

Results Messages

	id	customer_id	total_amount
1	390	241	15450.34
2	535	94	13322.29
3	732	181	13488.79
4	896	104	13724.16
5	972	226	15112.01
6	1066	64	13583.12
7	1112	212	13515.23
8	1202	58	14816.06



Check for Impossible or Suspicious Values

Negative or zero prices

```
SELECT * FROM products
WHERE price <= 0;
```

90 %

Results	Messages
id	name
description	price
category_id	supplier_id
sku	stock_quantity

Orders with zero or negative total

```
SELECT * FROM orders
WHERE total_amount <= 0;
```

10 %

Results	Messages
id	customer_id
order_date	total_amount
status	

Inventory movements with zero quantity

```
SELECT * FROM inventory_movements
WHERE quantity = 0;
```

%

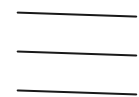
Results	Messages
id	product_id
quantity	movement_type
movement_date	

customers with no orders

```
select * from customers c where not exists (select 1 from orders o where c.id = o.customer_id)
```

90 %

Results	Messages
id	first_name
last_name	email
phone	address
registration_date	
1	48
Allison	Robinson
leemathew@example.net	+10000000048
03718 Andrew Motorway Apt. 522 West Patriciapo...	2025-01-28 14:20:47.000



Check Foreign Key Violations

Orders without valid customers

```
SELECT * FROM orders
WHERE customer_id NOT IN (SELECT id FROM customers);
```

1 %

Results Messages

id	customer_id	order_date	total_amount	status
----	-------------	------------	--------------	--------

Order details with invalid products

```
SELECT * FROM order_details
WHERE product_id NOT IN (SELECT id FROM products);
```

90 %

Results Messages

id	order_id	product_id	quantity	unit_price
----	----------	------------	----------	------------

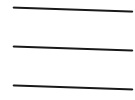
Check for Unused Products

```
SELECT * FROM products p
WHERE NOT EXISTS (SELECT 1 FROM order_details od WHERE od.product_id = p.id)
AND NOT EXISTS (SELECT 1 FROM wishlists w WHERE w.product_id = p.id)
AND NOT EXISTS (SELECT 1 FROM reviews r WHERE r.product_id = p.id);
```

90 %

Results Messages

id	name	description	price	category_id	supplier_id	sku	stock_quantity
----	------	-------------	-------	-------------	-------------	-----	----------------



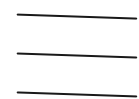
Data Analysis in SSMS

- Calculate the total sales revenue from all orders.
 - List the top 5 best-selling products by quantity sold.
 - Identify customers with the highest number of orders.
 - Generate an alert for products with stock quantities below 20 units
 - Determine the percentage of orders that used a discount.
 - Calculate the average rating for each product
- Compute the 30-day customer retention rate after their first purchase.
 - Recommend products frequently bought together with items in customer wishlists
 - Track inventory turnover trends using a 30-day moving average.
 - Identify customers who have purchased every product in a specific category.
 - Find pairs of products commonly bought together in the same order.
 - Calculate the time taken to deliver orders in days

Identify customers with the highest number of orders.

```
SELECT
    RANK() OVER (ORDER BY COUNT(o.id) DESC) AS rank_by_orders,
    c.id AS customer_id,
    c.first_name + ' ' + c.last_name AS customer_name,
    c.email,
    COUNT(o.id) AS num_orders,
    SUM(o.total_amount) AS total_spend,
    AVG(o.total_amount) AS avg_order_value,
    CAST(MIN(o.order_date) AS DATE) AS first_order_date,
    CAST(MAX(o.order_date) AS DATE) AS last_order_date
FROM customers c
JOIN orders o ON c.id = o.customer_id
GROUP BY c.id, c.first_name, c.last_name, c.email
ORDER BY num_orders DESC;
```

	rank_by_orders	customer_id	customer_name	email	num_orders	total_spend	avg_order_value	first_order_date	last_order_date
1	1	212	Ryan Chang	rachel30@example.org	73	302643.41	4145.800136	2025-01-03	2025-05-03
2	2	90	Suzanne Bennett	kyle00@example.net	68	297327.19	4372.458676	2025-01-01	2025-05-06
3	2	134	Robyn Reed	carsonaustin@example.net	68	307115.89	4516.410147	2025-01-01	2025-05-06
4	4	116	Zachary Aguirre	johnsonjennifer@example.com	66	329685.39	4995.233181	2025-01-01	2025-05-03
5	5	189	Mark Spence	smithheather@example.org	65	265418.79	4083.366000	2025-01-06	2025-05-06
6	6	190	Leslie Alvarado	jaredkelly@example.net	64	274766.12	4293.220625	2025-01-03	2025-05-07
7	6	188	Tiffany Boone	rebecca45@example.net	64	290029.65	4531.713281	2025-01-02	2025-05-07
8	6	91	Dana Farrell	shelly85@example.com	64	308903.76	4826.621250	2025-01-01	2025-05-06
9	6	55	John Perez	christianrobert@example.net	64	275604.08	4306.313750	2025-01-03	2025-05-07



Recommend products frequently bought together with items in customer wishlists

```
CREATE PROCEDURE sp_GetFrequentlyBoughtWithWishlistProducts
AS
BEGIN
    -- CTE for wishlisted products per customer
    WITH WishlistProducts AS (
        SELECT customer_id, product_id
        FROM wishlists
    ),

    -- Orders made by those customers
    CustomerOrders AS (
        SELECT o.customer_id, od.product_id
        FROM orders o
        INNER JOIN order_details od ON o.id = od.order_id
    ),

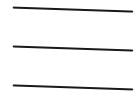
    -- Join to find frequently bought-together items
    ProductsBoughtWithWishlist AS (
        SELECT
            wp.product_id AS wishlist_product_id,
            co.product_id AS bought_product_id,
            COUNT(*) AS times_bought_together
        FROM WishlistProducts wp
        INNER JOIN CustomerOrders co
            ON wp.customer_id = co.customer_id
            AND wp.product_id <> co.product_id
        GROUP BY wp.product_id, co.product_id
    )

    -- Return with product names
    SELECT
        pbw.wishlist_product_id,
        wp.name AS wishlist_product_name,
        pbw.bought_product_id,
        bp.name AS bought_product_name,
        pbw.times_bought_together
    FROM ProductsBoughtWithWishlist pbw
    INNER JOIN products wp ON wp.id = pbw.wishlist_product_id
    INNER JOIN products bp ON bp.id = pbw.bought_product_id
    ORDER BY
        pbw.times_bought_together DESC;
END;
```

56 %

Results Messages

	wishlist_product_id	wishlist_product_name	bought_product_id	bought_product_name	times_bought_together
1	201	Streamlined tangible ability	568	Multi-channeled bottom-line orchestration	9
2	535	Customizable homogeneous installation	458	Up-sized multi-tasking productivity	8



Calculate the time taken to deliver orders in days.

```
-- Calculate the time taken to deliver orders in days.
SELECT
  o.id AS order_id,
  c.first_name + ' ' + c.last_name AS customer_name,
  o.order_date,
  s.shipping_date,
  DATEDIFF(day, o.order_date, s.shipping_date) AS delivery_days,
  CASE
    WHEN DATEDIFF(day, o.order_date, s.shipping_date) <= 3 Then 'Fast'
    WHEN DATEDIFF(day, o.order_date, s.shipping_date) <= 7 THEN 'Standard'
    ELSE 'Slow'
  END AS delivery_speed
from orders o
INNER JOIN customers c on o.customer_id = c.id
INNER JOIN shipping s on o.id = s.order_id
Where s.status = 'delivered'
Order by delivery_days DESC;
```

	order_id	customer_name	order_date	shipping_date	delivery_days	delivery_speed
1	8	Jason Lewis	2025-03-12 04:09:19.000	2025-03-13 04:09:19.000	1	Fast
2	9	Kimberly Wiley	2025-01-03 14:50:19.000	2025-01-04 14:50:19.000	1	Fast
3	17	Carly Duran	2025-01-24 00:10:42.000	2025-01-25 00:10:42.000	1	Fast
4	33	Sheila Cohen	2025-03-20 16:32:46.000	2025-03-21 16:32:46.000	1	Fast
5	34	Martin Thomas	2025-05-03 01:18:51.000	2025-05-04 01:18:51.000	1	Fast

Customer Features View



```
CREATE VIEW vw_CustomerFeatures AS
SELECT
  c.id AS customer_id,
  c.first_name,
  c.last_name,
  c.email,
  c.registration_date,
  DATEDIFF(DAY, c.registration_date, GETDATE()) AS customer_tenure_days,

  -- Order behavior
  COUNT(DISTINCT o.id) AS total_orders,
  SUM(o.total_amount) AS total_spend,
  CASE WHEN COUNT(DISTINCT o.id) > 0 THEN SUM(o.total_amount)/COUNT(DISTINCT o.id) ELSE 0 END AS avg_order_value,
  MAX(o.order_date) AS last_order_date,
  DATEDIFF(DAY, MAX(o.order_date), GETDATE()) AS days_since_last_order,

  -- Return behavior
  COUNT(DISTINCT r.id) AS total_returns,
  CASE WHEN COUNT(DISTINCT o.id) > 0 THEN CAST(COUNT(DISTINCT r.id) AS FLOAT)/COUNT(DISTINCT o.id) ELSE 0 END AS return_rate,

  -- Session behavior
  COUNT(DISTINCT cs.id) AS total_sessions,
  AVG(DATEDIFF(MINUTE, cs.session_start, cs.session_end)) AS avg_session_duration,

  -- Wishlist behavior
  COUNT(DISTINCT w.id) AS wishlist_items,

  -- Payment behavior
  COUNT(DISTINCT CASE WHEN p.status = 'completed' THEN p.id END) AS successful_payments,
  COUNT(DISTINCT CASE WHEN p.status = 'failed' THEN p.id END) AS failed_payments,

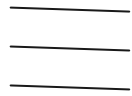
  -- Calculate customer value metrics
  SUM(o.total_amount) / NULLIF(DATEDIFF(DAY, MIN(o.order_date), GETDATE()), 0) * 30 AS monthly_revenue,
  DATEDIFF(DAY, MIN(o.order_date), MAX(o.order_date)) / NULLIF(COUNT(DISTINCT o.id) - 1, 0) AS avg_days_between_orders
FROM
  customers c
  LEFT JOIN orders o ON c.id = o.customer_id
  LEFT JOIN returns r ON o.id = r.order_id
  LEFT JOIN customer_sessions cs ON c.id = cs.customer_id
  LEFT JOIN wishlists w ON c.id = w.customer_id
  LEFT JOIN payments p ON c.id = p.customer_id
GROUP BY
  c.id, c.first_name, c.last_name, c.email, c.registration_date;
```

Customer Category Pivot View



-- This creates a wide table with categories as columns

```
CREATE VIEW vw_CustomerCategoryPivot AS
WITH CategoryPurchases AS (
    SELECT
        o.customer_id,
        p.category_id,
        c.name AS category_name,
        COUNT(DISTINCT o.id) AS purchase_count,
        SUM(od.quantity) AS items_purchased
    FROM
        orders o
    JOIN order_details od ON o.id = od.order_id
    JOIN products p ON od.product_id = p.id
    JOIN categories c ON p.category_id = c.id
    GROUP BY
        o.customer_id, p.category_id, c.name
)
SELECT
    cp.customer_id,
    MAX(CASE WHEN cp.category_name = 'Adult' THEN cp.purchase_count ELSE 0 END) AS Adult_purchases,
    MAX(CASE WHEN cp.category_name = 'Almost' THEN cp.purchase_count ELSE 0 END) AS Almost_purchases,
    MAX(CASE WHEN cp.category_name = 'Base' THEN cp.purchase_count ELSE 0 END) AS Base_purchases,
    MAX(CASE WHEN cp.category_name = 'Born' THEN cp.purchase_count ELSE 0 END) AS Born_purchases,
    MAX(CASE WHEN cp.category_name = 'Carry' THEN cp.purchase_count ELSE 0 END) AS Carry_purchases,
    MAX(CASE WHEN cp.category_name = 'Choose' THEN cp.purchase_count ELSE 0 END) AS Choose_purchases,
    MAX(CASE WHEN cp.category_name = 'Energy' THEN cp.purchase_count ELSE 0 END) AS Energy_purchases,
    MAX(CASE WHEN cp.category_name = 'Fire' THEN cp.purchase_count ELSE 0 END) AS Fire_purchases,
    MAX(CASE WHEN cp.category_name = 'Free' THEN cp.purchase_count ELSE 0 END) AS Free_purchases,
    MAX(CASE WHEN cp.category_name = 'Game' THEN cp.purchase_count ELSE 0 END) AS Game_purchases,
    MAX(CASE WHEN cp.category_name = 'Glass' THEN cp.purchase_count ELSE 0 END) AS Glass_purchases,
    MAX(CASE WHEN cp.category_name = 'Half' THEN cp.purchase_count ELSE 0 END) AS Half_purchases,
    MAX(CASE WHEN cp.category_name = 'Himself' THEN cp.purchase_count ELSE 0 END) AS Himself_purchases,
    MAX(CASE WHEN cp.category_name = 'Hold' THEN cp.purchase_count ELSE 0 END) AS Hold_purchases,
    MAX(CASE WHEN cp.category_name = 'Large' THEN cp.purchase_count ELSE 0 END) AS Large_purchases,
    MAX(CASE WHEN cp.category_name = 'Letter' THEN cp.purchase_count ELSE 0 END) AS Letter_purchases,
    MAX(CASE WHEN cp.category_name = 'Marriage' THEN cp.purchase_count ELSE 0 END) AS Marriage_purchases,
    MAX(CASE WHEN cp.category_name = 'Mission' THEN cp.purchase_count ELSE 0 END) AS Mission_purchases,
    MAX(CASE WHEN cp.category_name = 'Onto' THEN cp.purchase_count ELSE 0 END) AS Onto_purchases,
    MAX(CASE WHEN cp.category_name = 'Or' THEN cp.purchase_count ELSE 0 END) AS Or_purchases,
    MAX(CASE WHEN cp.category_name = 'Over' THEN cp.purchase_count ELSE 0 END) AS Over_purchases,
    MAX(CASE WHEN cp.category_name = 'Part' THEN cp.purchase_count ELSE 0 END) AS Part_purchases,
    MAX(CASE WHEN cp.category_name = 'Program' THEN cp.purchase_count ELSE 0 END) AS Program_purchases,
    MAX(CASE WHEN cp.category_name = 'Recent' THEN cp.purchase_count ELSE 0 END) AS Recent_purchases,
    MAX(CASE WHEN cp.category_name = 'Remember' THEN cp.purchase_count ELSE 0 END) AS Remember_purchases,
    MAX(CASE WHEN cp.category_name = 'Share' THEN cp.purchase_count ELSE 0 END) AS Share_purchases,
    MAX(CASE WHEN cp.category_name = 'Speech' THEN cp.purchase_count ELSE 0 END) AS Speech_purchases,
    MAX(CASE WHEN cp.category_name = 'Table' THEN cp.purchase_count ELSE 0 END) AS Table_purchases,
    MAX(CASE WHEN cp.category_name = 'Who' THEN cp.purchase_count ELSE 0 END) AS Who_purchases,
    MAX(CASE WHEN cp.category_name = 'Window' THEN cp.purchase_count ELSE 0 END) AS Window_purchases,
    COUNT(DISTINCT cp.category_id) AS unique_categories_purchased
FROM
    CategoryPurchases cp
GROUP BY
    cp.customer_id;
```



Full Report



```
CREATE VIEW vw_CustomerChurnDataset AS
WITH ChurnDefinition AS (
    SELECT
        c.id AS customer_id,
        CASE WHEN MAX(o.order_date) IS NULL OR DATEDIFF(DAY, MAX(o.order_date), GETDATE()) > 90 THEN 1 ELSE 0 END AS is_churned
    FROM
        customers c
    LEFT JOIN orders o ON c.id = o.customer_id
    GROUP BY
        c.id
)
SELECT
    cf.customer_id,
    cf.customer_tenure_days,
    cf.total_orders,
    cf.total_spend,
    cf.avg_order_value,
    cf.days_since_last_order,
    cf.return_rate,
    cf.total_sessions,
    cf.avg_session_duration,
    cf.wishlist_items,
    cf.successful_payments,
    cf.failed_payments,
    cf.monthly_revenue,
    cf.avg_days_between_orders,
    ccp.unique_categories_purchased,
    ccp.Adult_purchases,
    ccp.Almost_purchases,
    ccp.Base_purchases,
    ccp.Born_purchases,
    ccp.Carry_purchases,
    ccp.Choose_purchases,
    ccp.Energy_purchases,
    ccp.Fire_purchases,
    ccp.Free_purchases,
    ccp.Game_purchases,
    ccp.Glass_purchases,
    ccp.Half_purchases,
    ccp.Himself_purchases,
    ccp.Hold_purchases,
    ccp.Large_purchases,
    ccp.Letter_purchases,
    ccp.Marriage_purchases,
    ccp.Mission_purchases,
    ccp.Onto_purchases,
    ccp.Or_purchases,
    ccp.Over_purchases,
    ccp.Part_purchases,
    ccp.Program_purchases,
    ccp.Recent_purchases,
    ccp.Remember_purchases,
    ccp.Share_purchases,
    ccp.Speech_purchases,
    ccp.Table_purchases,
    ccp.Who_purchases,
    ccp.Window_purchases,
    -- Target variable
    cd.is_churned
FROM
    vw_CustomerFeatures cf
JOIN ChurnDefinition cd ON cf.customer_id = cd.customer_id
LEFT JOIN vw_CustomerCategoryPivot ccp ON cf.customer_id = ccp.customer_id;

-- VIEW THE FULL REPORT
SELECT top 5 * FROM vw_CustomerChurnDataset
ORDER BY customer_id;
```

Calculate the total sales revenue from all orders

```
SELECT
    ROUND(SUM(od.quantity * od.unit_price), 2) AS total_sales_revenue,
    COUNT(DISTINCT o.id) AS total_orders,
    COUNT(DISTINCT o.customer_id) AS unique_customers,
    ROUND(SUM(od.quantity * od.unit_price) / NULLIF(COUNT(DISTINCT o.id), 0), 2) AS avg_order_value
FROM
    orders o
JOIN
    order_details od ON o.id = od.order_id
WHERE
    o.status != 'cancelled';
```

1 %

Results Messages

	total_sales_revenue	total_orders	unique_customers	avg_order_value
1	36076243.32	8188	248	4405.990000

Calculate the total sales revenue from all orders

```
-- Cancelled orders: total amount and count
SELECT
  COUNT(DISTINCT o.id) AS cancelled_order_count,
  SUM(od.quantity * od.unit_price) AS cancelled_order_revenue
FROM
  orders o
JOIN
  order_details od ON o.id = od.order_id
WHERE
  o.status = 'cancelled';

SELECT
  ROUND(SUM(od.quantity * od.unit_price), 2) AS total_sales_revenue,
  COUNT(DISTINCT o.id) AS total_orders
FROM
  orders o
JOIN
  order_details od ON o.id = od.order_id;
```

3 %

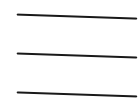
	cancelled_order_count	cancelled_order_revenue
1	2012	8867722.70

	total_sales_revenue	total_orders
1	44943966.02	10200

List the top 5 best-selling products by quantity sold. (Ranked by units sold, with revenue)

```
SELECT TOP 5
    p.id AS product_id,
    p.name AS product_name,
    SUM(od.quantity) AS total_units_sold,
    ROUND(SUM(od.quantity * od.unit_price), 2) AS total_revenue
FROM
    order_details od
JOIN
    products p ON od.product_id = p.id
JOIN
    orders o ON od.order_id = o.id
WHERE
    o.status != 'cancelled'
GROUP BY
    p.id, p.name
ORDER BY
    total_units_sold DESC;
```

	product_id	product_name	total_units_sold	total_revenue
1	216	Up-sized interactive time-frame	214	132288.38
2	591	Vision-oriented scalable archive	208	165917.44
3	499	Versatile holistic help-desk	205	124541.60
4	222	Compatible optimal knowledgebase	204	58301.16
5	531	Vision-oriented mission-critical application	202	57163.98



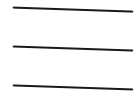
Compute the 30-day customer retention rate after their first purchase.

```
WITH
-- First purchase date per valid customer
FirstPurchase AS (
  SELECT
    customer_id,
    MIN(order_date) AS first_purchase_date
  FROM
    orders
  WHERE
    status != 'cancelled'
    AND customer_id NOT LIKE 'test%' -- Exclude test accounts
  GROUP BY
    customer_id
),
-- Customers who made a follow-up purchase within 30 days
RetainedCustomers AS (
  SELECT DISTINCT
    o.customer_id
  FROM
    orders o
  JOIN
    FirstPurchase fp ON o.customer_id = fp.customer_id
  WHERE
    o.order_date > fp.first_purchase_date -- Exclude same-day purchases
    AND o.order_date <= DATEADD(day, 30, fp.first_purchase_date)
    AND o.status != 'cancelled'
)
-- Final retention rate calculation
SELECT
  COUNT(DISTINCT fp.customer_id) AS total_customers,
  COUNT(DISTINCT rc.customer_id) AS retained_customers,
  ROUND(COUNT(DISTINCT rc.customer_id) * 100.0 /
    NULLIF(COUNT(DISTINCT fp.customer_id), 0), 1) AS retention_rate_pct
FROM
  FirstPurchase fp
LEFT JOIN
  RetainedCustomers rc ON fp.customer_id = rc.customer_id;
```

56 %

Results Messages

	total_customers	retained_customers	retention_rate_pct
1	248	227	91.500000000000



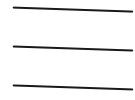
Calculate the average rating for each product.

```
--Calculate the average rating for each product.  
select  
  p.id as Product_id,  
  p.name as Product_name,  
  avg(r.rating) as average_rating  
from products p  
inner join reviews r  
on p.id=r.product_id  
group by p.name ,p.id
```

91 %

Results Messages

	Product_id	Product_name	average_rating
1	1	Enterprise-wide stable synergy	2
2	2	Synergized uniform contingency	1
3	3	Function-based well-modulated intranet	3
4	4	Monitored bifurcated database	2
5	5	Versatile modular info-mediaries	3
6	6	Multi-channeled foreground adapter	3
7	7	Fundamental grid-enabled definition	3



Identify customers who have purchased every product in a specific category.

```
--Identify customers who have purchased every product in a specific category
SELECT
  c.id AS customer_id,
  c.first_name,
  c.last_name,
  cat.id AS category_id,
  cat.name AS category_name
FROM
  customers c
  JOIN orders o ON c.id = o.customer_id
  JOIN order_details od ON o.id = od.order_id
  JOIN products p ON od.product_id = p.id
  JOIN categories cat ON p.category_id = cat.id
WHERE
  cat.name = 'Window' -- Replace with the desired category name
GROUP BY
  c.id, c.first_name, c.last_name, cat.id, cat.name
```

75 %

Results Messages

	customer_id	first_name	last_name	category_id	category_name
1	1	Loretta	Weaver	8	Window
2	2	Jennifer	Kelley	8	Window
3	3	Michael	Mcintosh	8	Window
4	4	James	Nguyen	8	Window
5	5	Sheila	Cohen	8	Window

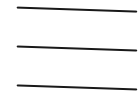


Find pairs of products commonly bought together in the same order.

```
-- Find pairs of products commonly bought together in the same order.

SELECT
    od1.product_id AS product_1,
    od2.product_id AS product_2,
    COUNT(*) AS times_bought_together
FROM
    order_details od1
JOIN
    order_details od2
    ON od1.order_id = od2.order_id AND od1.product_id < od2.product_id
GROUP BY
    od1.product_id, od2.product_id
ORDER BY
    times_bought_together DESC;
```

	product_1	product_2	times_bought_together
1	282	341	5
2	488	595	4
3	179	440	4
4	244	487	4
5	293	354	4
6	137	249	4
7	348	441	4



Generate an alert for products with stock quantities below 20 units

SQLQueries.sql - ...D2KEN0\HP G4 (65))*

SELECT

RANK() OVER (ORDER BY p.stock_quantity ASC) AS stock_rank,

p.id AS product_id,

p.name AS product_name,

p.stock_quantity,

c.name AS category_name,

s.name AS supplier_name,

p.price,

'Low' AS stock_status

FROM products p

LEFT JOIN categories c ON p.category_id = c.id

LEFT JOIN suppliers s ON p.supplier_id = s.id

WHERE stock_quantity < 20

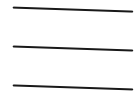
ORDER BY p.stock_quantity ASC;

90 %

Results

Messages

	stock_rank	product_id	product_name	stock_quantity	category_name	supplier_name	price	stock_status
1	1	84	Upgradable fresh-thinking model	0	Adult	Davies, Jackson and Randolph	69.27	Low
2	2	12	Advanced modular capacity	3	Part	Freeman-Valenzuela	716.47	Low
3	3	15	Focused radical protocol	6	Adult	Wright-Davis	228.16	Low
4	3	3	Function-based well-modulated intranet	6	Half	Figueroa PLC	712.42	Low
5	3	20	Reverse-engineered fresh-thinking success	6	Half	Cox-Williams	306.61	Low
6	3	25	Exclusive actuating open system	6	Adult	Wright-Davis	777.38	Low
7	3	33	Distributed empowering leverage	6	Recent	Crawford, Adams and Savage	210.22	Low
8	3	100	Reactive tertiary moratorium	6	Window	Osborn-Stewart	437.07	Low
9	9	71	Distributed fault-tolerant process improve...	7	Half	Sanders Ltd	864.49	Low
10	9	72	Cross-platform fault-tolerant secured line	7	Half	Crawford, Adams and Savage	389.19	Low
11	9	34	Configurable optimizing extranet	7	Hold	Williams Group	561.51	Low



Determine the percentage of orders that used a discount.

```
--Determine the percentage of orders that used a discount

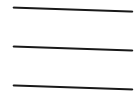
WITH DiscountedOrders AS (
  SELECT DISTINCT od.order_id
  FROM order_details od
  JOIN discounts d ON od.product_id = d.product_id
  JOIN orders o ON od.order_id = o.id
  WHERE
    d.is_active = 1
)

-- Final percentage calculation
SELECT
  COUNT(DISTINCT do.order_id) AS discounted_orders,
  (SELECT COUNT(DISTINCT id) FROM orders) AS total_orders,
  ROUND(
    COUNT(DISTINCT do.order_id) * 100.0 /
    NULLIF((SELECT COUNT(DISTINCT id) FROM orders), 0), 2
  ) AS discount_usage_percentage
FROM
  DiscountedOrders do;
```

91 %

Results Messages

	discounted_orders	total_orders	discount_usage_percentage
1	637	10200	6.250000000000



Track inventory turnover trends using a 30-day moving average.

```
--Track inventory turnover trends using a 30-day moving average

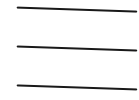
WITH DailySales AS (
  SELECT
    product_id,
    CAST(movement_date AS DATE) AS sale_date,
    SUM(-quantity) AS daily_units_sold -- Convert negative to positive sales
  FROM
    inventory_movements
  WHERE
    movement_type = 'sale'
  GROUP BY
    product_id, CAST(movement_date AS DATE)
)

SELECT
  product_id,
  sale_date,
  daily_units_sold,
  AVG(daily_units_sold * 1.0) OVER (
    PARTITION BY product_id
    ORDER BY sale_date
    ROWS BETWEEN 29 PRECEDING AND CURRENT ROW
  ) AS moving_avg_30day
FROM
  DailySales
ORDER BY
  product_id, sale_date;
```

58 %

Results Messages

	product_id	sale_date	daily_units_sold	moving_avg_30day
1	1	2025-02-11	1	1.000000
2	1	2025-02-20	3	2.000000
3	1	2025-03-16	1	1.666666

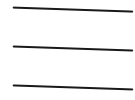


Recommendation

Several products consistently receive low ratings (3 or below).

Recommendations:

- o Focus improvement efforts on products with ratings ≤ 3 .
- o Run surveys for low-rated products to gather detailed feedback.
- o Send follow-up emails asking happy customers to leave reviews.
- o Offer small incentives (discounts, loyalty points) for leaving a review.



1. Launch a Loyalty Program

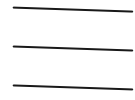
- Reward top customers with points, discounts, or exclusive benefits.
- Encourage repeat purchases and increase customer retention.

2. Create VIP Customer Tiers

- Introduce “Gold” or “Platinum” levels for frequent buyers.
- Offer perks like early access, premium support, or free shipping.

3. Collect Direct Feedback

- Engage top customers through surveys or feedback forms.
- Use insights to enhance user experience and service quality.



4. Implement Smart Product Bundles

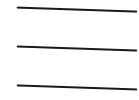
- Create dynamic bundles featuring wishlist items and their frequently bought companions.
- Offer bundle discounts to encourage larger cart sizes.

5. Cart Suggestions Based on Wishlist

- At checkout, recommend products commonly bought with items from the customer's wishlist.
- Increase average order value with timely, relevant prompts.

6. Targeted Ads Based on Wishlist Associations

- Retarget users with ads showing complementary products.



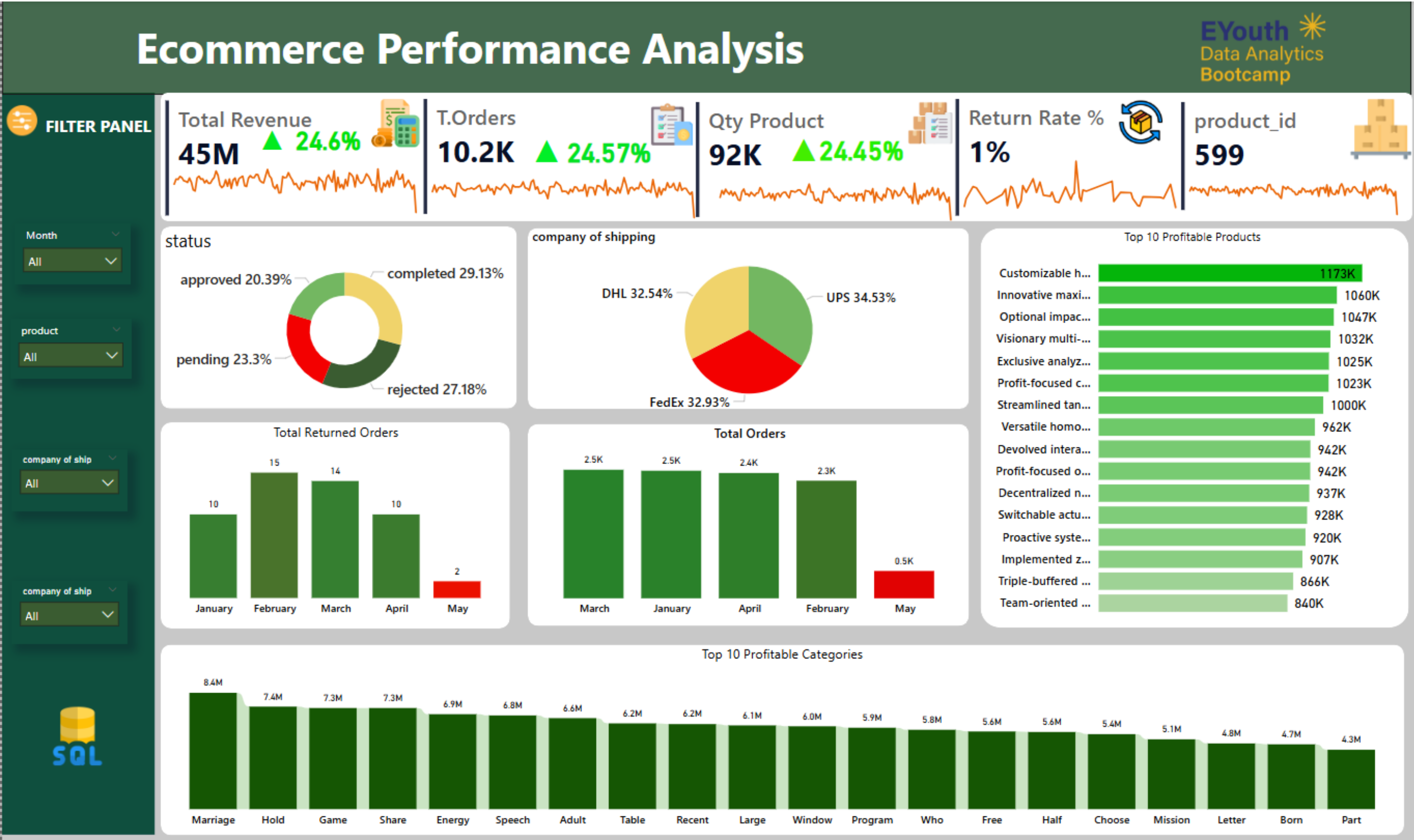
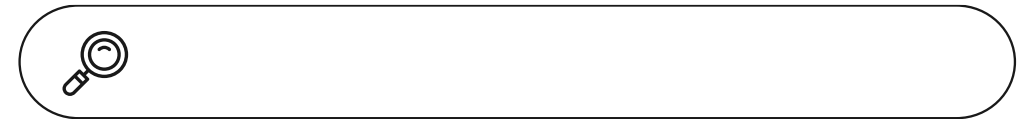
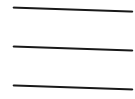
Advanced Data Analysis in Power BI

Visual-Driven Insights

- Sales Performance: Revenue trends, AOV, discount impact.
- Product Analysis: Best/worst performers, returns, stock alerts.
- Customer Segmentation: Top customers, retention, lifetime value.

Tools & Calculations Used

- DAX for custom measures (CLV, discount effect, YoY growth).
- Power Query for data modeling and cleaning.



Ecommerce Performance Analysis

EYouth
Data Analytics
Bootcamp

FILTER PANEL

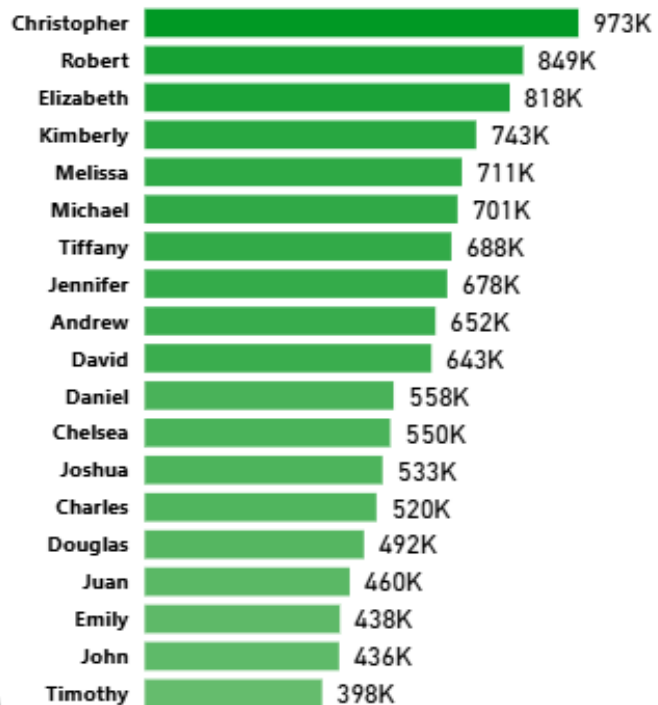
Month
All

fullName
All

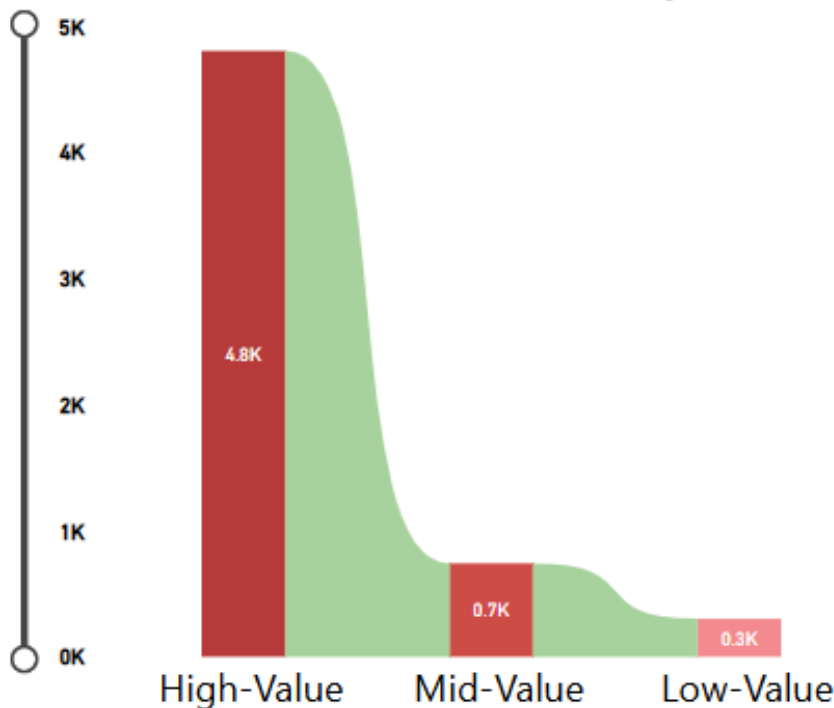
company of ship
All



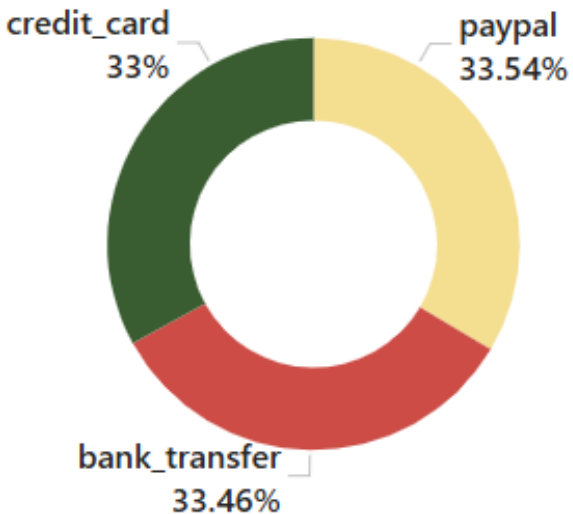
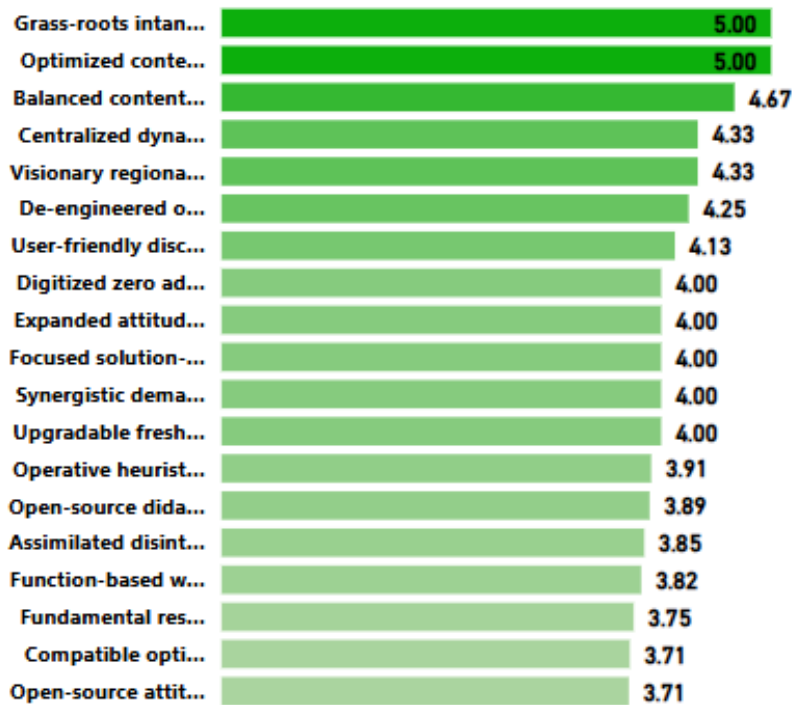
Top Profitable Customers



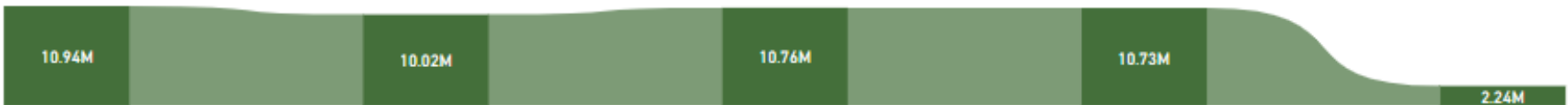
customer classification by clv



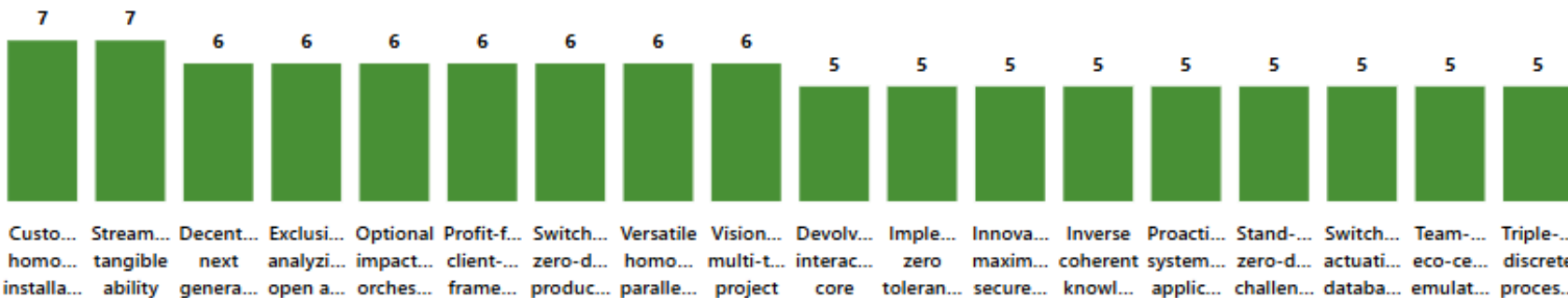
Average of rating by name



Monthly Sales Performance



Wishlist Frequency per Product



Ecommerce Performance Analysis

EYouth
Data Analytics
Bootcamp

FILTER PANEL

Month

All

status

All

classification

All

company of ship

All

Total Revenue

45M



T.Orders

10.2K



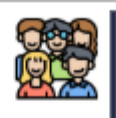
Qty Product

92K



Total Customers

250

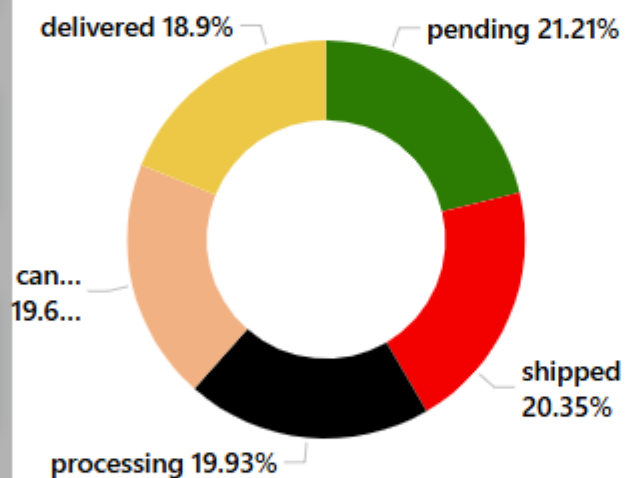


Count of product_id

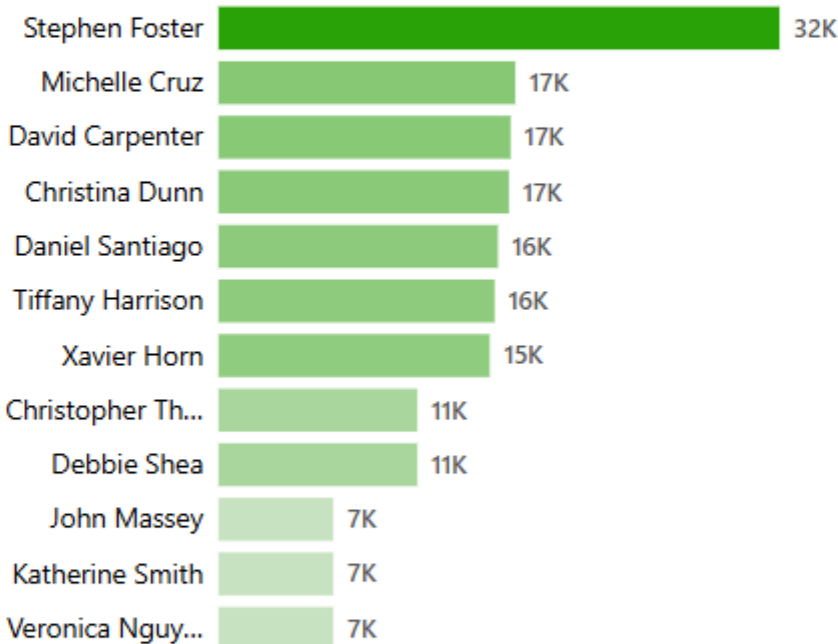
599



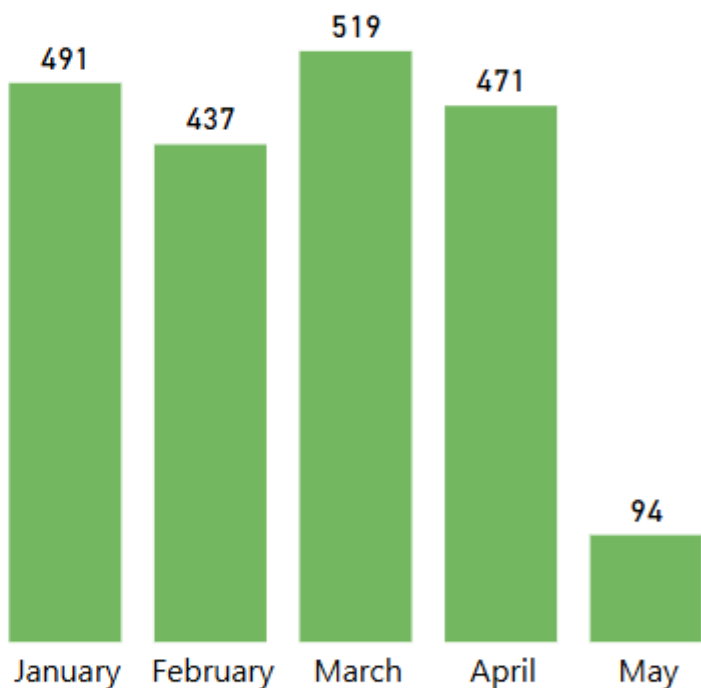
status per costemer



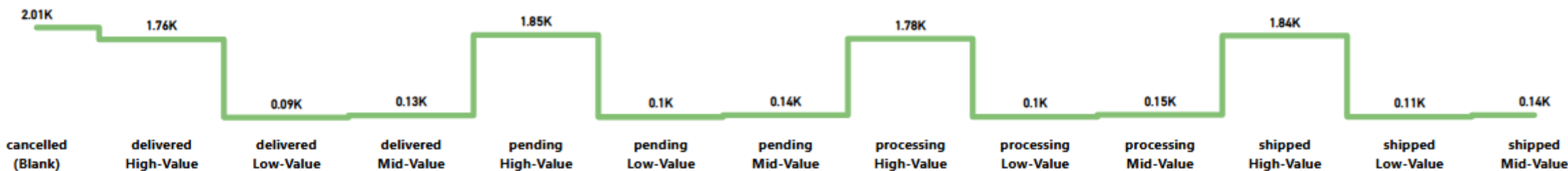
Top 10 payment Custemor

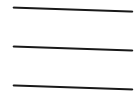


Cancelled Orders



Classification of shipping per level of customers





Ecommerce Performance Analysis

EYouth
Data Analytics
Bootcamp

FILTER PANEL

Month

All

product

All

fullName

All

company of ship

All



Total Revenue
45M



T.Orders
10.2K



Qty Product
92K



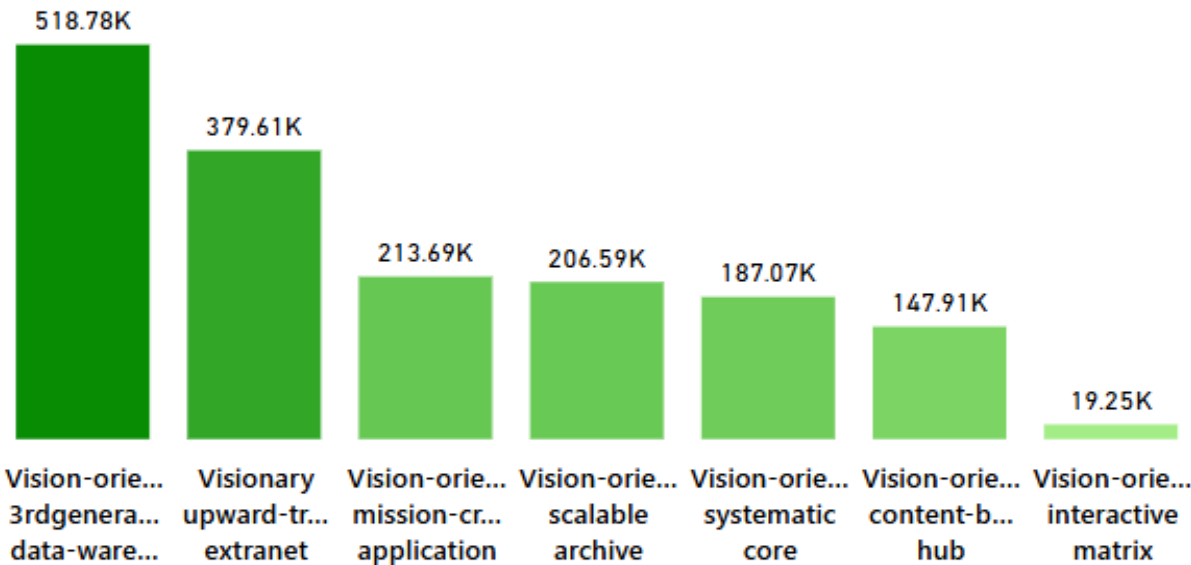
T.Supplies
70



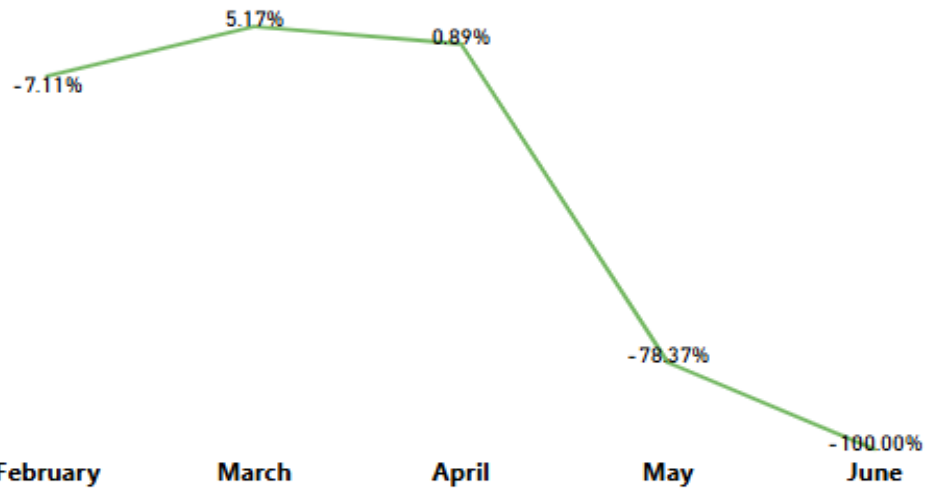
count of products
599



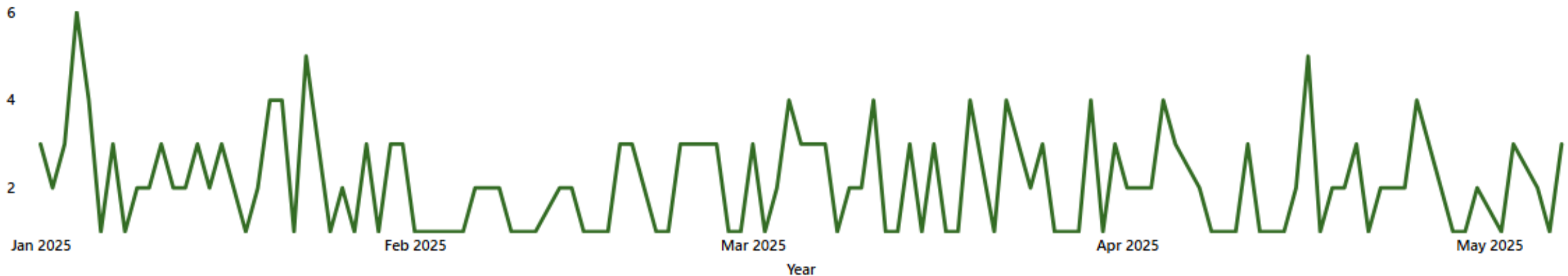
TOP 10 product for High level CustomerRevenue

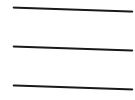


Month-Over-Month Growth%



Customers over the Year





Ecommerce Performance Analysis

EYouth
Data Analytics
Bootcamp



FILTER PANEL

Month

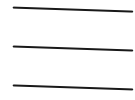
All

product

All



Product id	Category name	Product Name	Sum of stock_quantity	Low Inventory Status	Total Revenue	T.Orders	Returns Value	T.Reviews	Avg Rating
535	Large	Customizable homogeneous installation	186	High	1173K	267.0	10,767.09	10	3
585	Speech	Innovative maximized secured line	240	Overstock	1060K	250.0		19	3
131	Who	Optional impactful orchestration	212	Overstock	1047K	237.0	5,607.16	43	3
244	Marriage	Visionary multi-tasking project	208	Overstock	1032K	240.0	952.17	42	3
290	Large	Exclusive analyzing open architecture	220	Overstock	1025K	224.0	6,380.16	14	4
162	Free	Profit-focused client-driven framework	139	High	1023K	236.0		12	3
201	Speech	Streamlined tangible ability	150	High	1000K	237.0	11,431.15	11	3
102	Who	Versatile homogeneous parallelism	155	High	962K	231.0	952.17	43	3
192	Free	Devolved interactive core	184	High	942K	205.0	1,206.50	18	3
463	Share	Profit-focused optimizing product	130	High	942K	208.0	4,199.38	14	2
434	Speech	Decentralized next generation structure	177	High	937K	220.0	17,440.61	11	3
267	Mission	Switchable actuating database	142	High	928K	213.0	5,781.60	50	3
565	Free	Proactive systematic application	162	High	920K	204.0	4,972.38	7	3
582	Game	Implemented zero tolerance leverage	215	Overstock	907K	191.0	10,767.09	14	3
167	Hold	Triple-buffered discrete process improvement	178	High	866K	202.0		31	3
324	Free	Team-oriented eco-centric emulation	203	Overstock	840K	182.0	18,241.69	17	2
598	Energy	Stand-alone zero-defect	154	High	822K	191.0	7,474.60	30	3
Total			91881		44698K	10200.0	241,959.39	14196	3



Key KPIs

- Total Revenue: 45M (+24.6%)
- Net Revenue: 44.46M (+26.6%)
- Total Orders: 10.2K (+24.57%)
- Quantity Sold: 92K (+24.45%)
- Return Rate: 1% (very low, indicates high customer satisfaction)
- Products Analyzed: 599

Returns Analysis

Completed: 29.13%

Rejected: 27.18% (high – requires review)

Pending: 23.3%

Approved: 20.39%

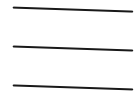
Returns peaked in February and March; dropped sharply in May.

Shipping Company Distribution

UPS: 34.53%

FedEx: 32.93%

DHL: 32.54%



Top Products & Categories

Top product: 'Customizable homogeneous installation' – Revenue: 1.17M

Most categories range between 6M–8.4M in revenue.

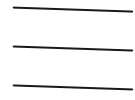
Inventory Observations

Many overstocked items are high-sellers – potential supply chain optimization. Products with high return value need attention (e.g., 'Team-oriented eco-centric emulation' – 18K+ return value).

Critical Drop in May

Orders dropped from average of ~2.4K to 0.5K – a 75%+ decrease. This is a red flag and could indicate marketing, technical, or supply issues.

Returns and reviews also dropped accordingly.



Insights & Recommendations

Sales Strategy

- Focus marketing on months with high AOV.
- Expand top-selling categories.

Inventory Planning

- Restock high-rated low-stock items.
- Remove long-idle products.

Customer Retention

- Target frequent discount users with loyalty programs.
- Win-back campaigns for dormant customers.

localhost:8084/#

Open Debugger

Vanna.AI

Functions

Training Data

+ New question

Show top 5 best-selling products

What is the total revenue and count of cancelled orders?

total revenue from cancelled orders

total revenue without cancelled orders

Which customers have purchased products in the 'Window' category?

Calculate the average rating for each product.

show top 5 best selling products

Calculate total revenue from only cancelled orders and show the top 5 best selling products.

Calculate total revenue from only cancelled orders and show the top 5 best selling products, and also

Welcome to Vanna.AI

Your AI-powered copilot for SQL queries.

You

show top 5 products by sales

Thinking...

Ask me a question about your data that I can turn into SQL.

localhost:8084/#

Vanna.AI

Functions

Training Data

New question

Show top 5 best-selling products

What is the total revenue and count of cancelled orders?

total revenue from cancelled orders

total revenue without cancelled orders

Which customers have purchased products in the 'Window' category?

Calculate the average rating for each product.

show top 5 best selling products

Calculate total revenue from only cancelled orders and show the top 5 best selling products.

Calculate total revenue from only cancelled orders and show the top 5 best selling products, and also

✓

PRODUCT_ID	PRODUCT_NAME	TOTAL_UNITS_SOLD
216	Up-sized interactive time-frame	214
591	Vision-oriented scalable archive	208
499	Versatile holistic help-desk	205
222	Compatible optimal knowledgebase	204
531	Vision-oriented mission-critical application	202

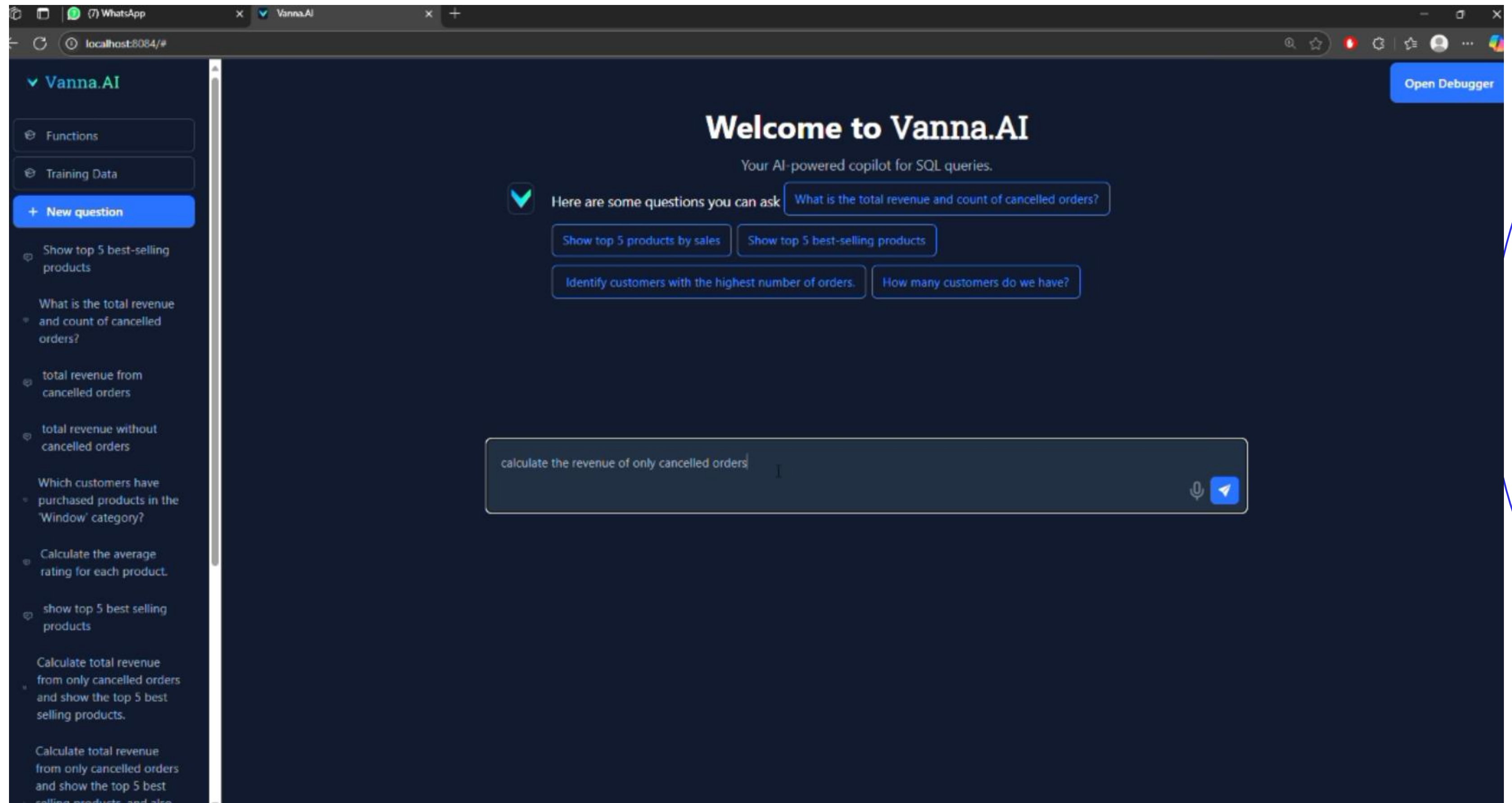
CSVDownload

✓

units_sold

product_id

Ask me a question about your data that I can turn into SQL.



Vanna.AI

Functions

Training Data

+ New question

Show top 5 best-selling products

What is the total revenue and count of cancelled orders?

total revenue from cancelled orders

total revenue without cancelled orders

Which customers have purchased products in the 'Window' category?

Calculate the average rating for each product.

show top 5 best selling products

Calculate total revenue from only cancelled orders and show the top 5 best selling products.

Calculate total revenue from only cancelled orders and show the top 5 best selling products, and also

DescriptionSQLtemplate

What is the total revenue and count of orders with status {order_status}?

✓

CANCELLED_ORDER_COUNT	CANCELLED_ORDER_REVENUE
2012	8867722.7

CSVDownload

✓

The data indicates that there were 2,012 cancelled orders, which generated a total revenue of approximately \$8,867,720.

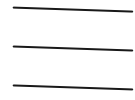
Play

✓

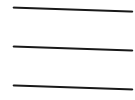
Were the results correct?

YouNo

Ask me a question about your data that I can turn into SQL.

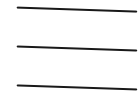


Machine Learning



Objective:

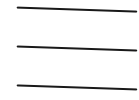
To predict trends and automate analysis using SQL Server data and modern Python-based ML tools. This documentation expands upon the technical implementation, tools used, and insights derived



4.1 Automated Data Cleaning

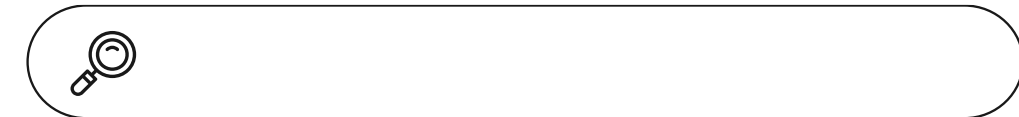
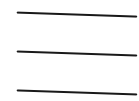
Tools & Definitions:

- Mito: A low-code spreadsheet-like interface integrated with Jupyter that allows for rapid data wrangling and generates Python code behind the scenes.
- Pandas: A powerful Python data analysis library used for handling structured data (DataFrames).
- pyodbc: A Python module for connecting to ODBC-compliant databases, like SQL Server.



Process:

1. Established a secure connection to SQL Server using pyodbc and extracted the following relational tables:
 - orders, customers, products, categories, order_details, reviews
2. Tables were loaded into a Python dictionary for modular and programmatic access.
3. Cleaning operations:
 - Filled missing parent_id fields in the categories table with the median.
 - Converted data types for date fields (order_date, registration_date) and price fields (unit_price, total_price).
4. Hierarchical enhancement:
 - Used recursive merging to represent parent-child relationships in categories.
 - Constructed a visual graph of category relationships using networkx.



Insight: Enhanced relational integrity and data quality, setting a strong foundation for downstream modeling and analytics.

HomeInsertDataFormulasCode

id | id

	id	customer_id	order_date	total_amount	status
	int	int	date	float	str
0	1	13	2025-04-05 09:06:50	521.85	shipped
1	2	15	2025-03-05 08:27:50	436.91	processing
2	3	9	2025-04-21 21:19:19	7,711.55	cancelled
3	4	19	2025-04-15 02:16:41	7,952.82	cancelled
4	5	31	2025-02-11 17:30:12	2,785.65	cancelled

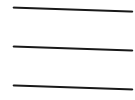
Unsupported Enviornment

Mito only supports JupyterLab and Jupyter Notebook, not wherever this is.

To install Mito in JupyterLab and Jupyter Notebook, follow our [installation instructions](#).

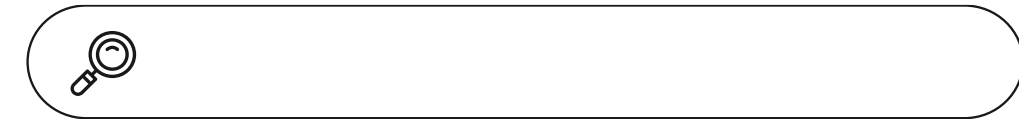
If you are still receiving this error message, join our slack to get support!

Help



Exploratory Data Analysis (EDA):

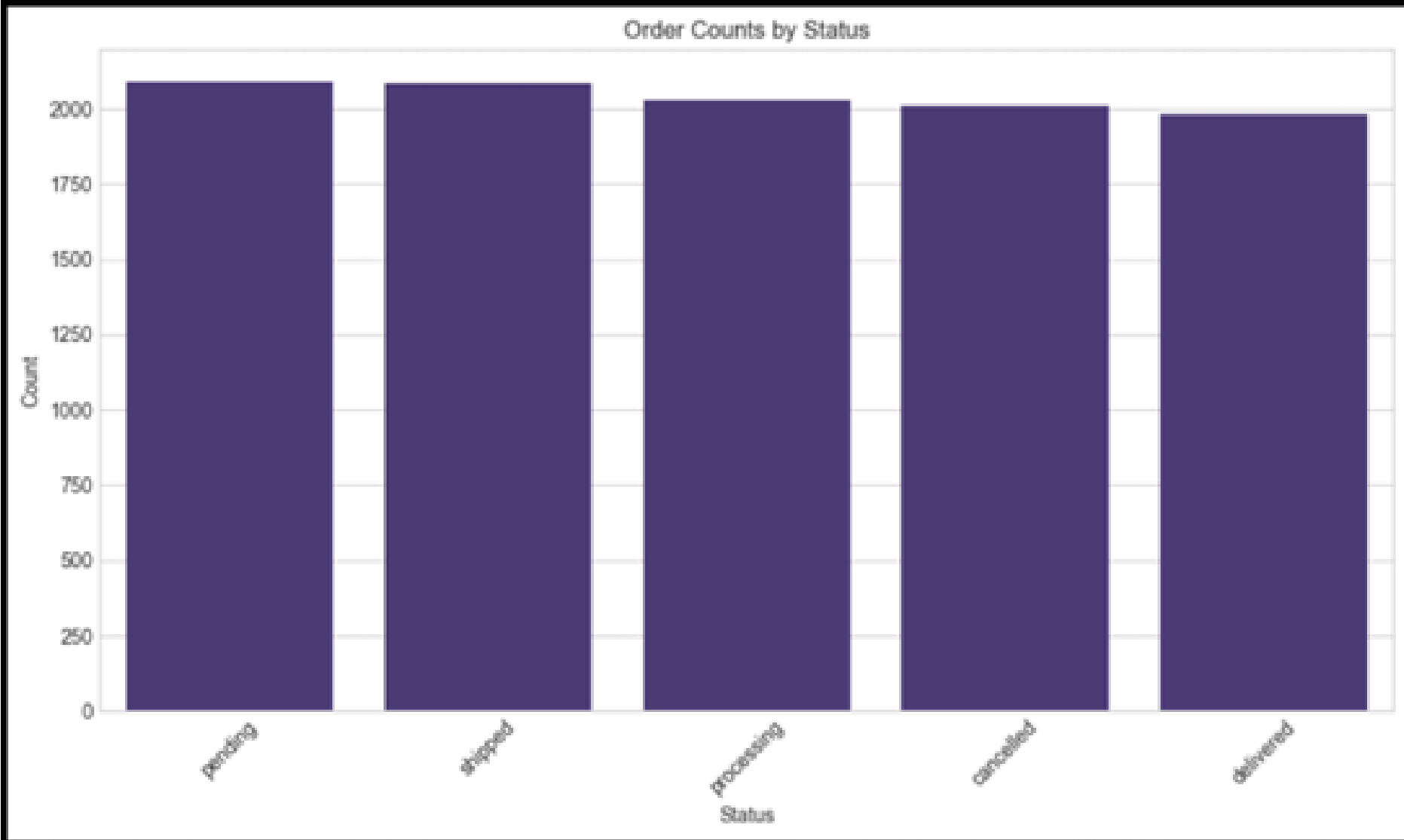
- **Generated visualizations to analyze:**
 - Order distribution by status and date
 - Product price and review rating histograms
 - Monthly revenue trends and customer acquisition timeline
 - Category-level sales and top products by revenue

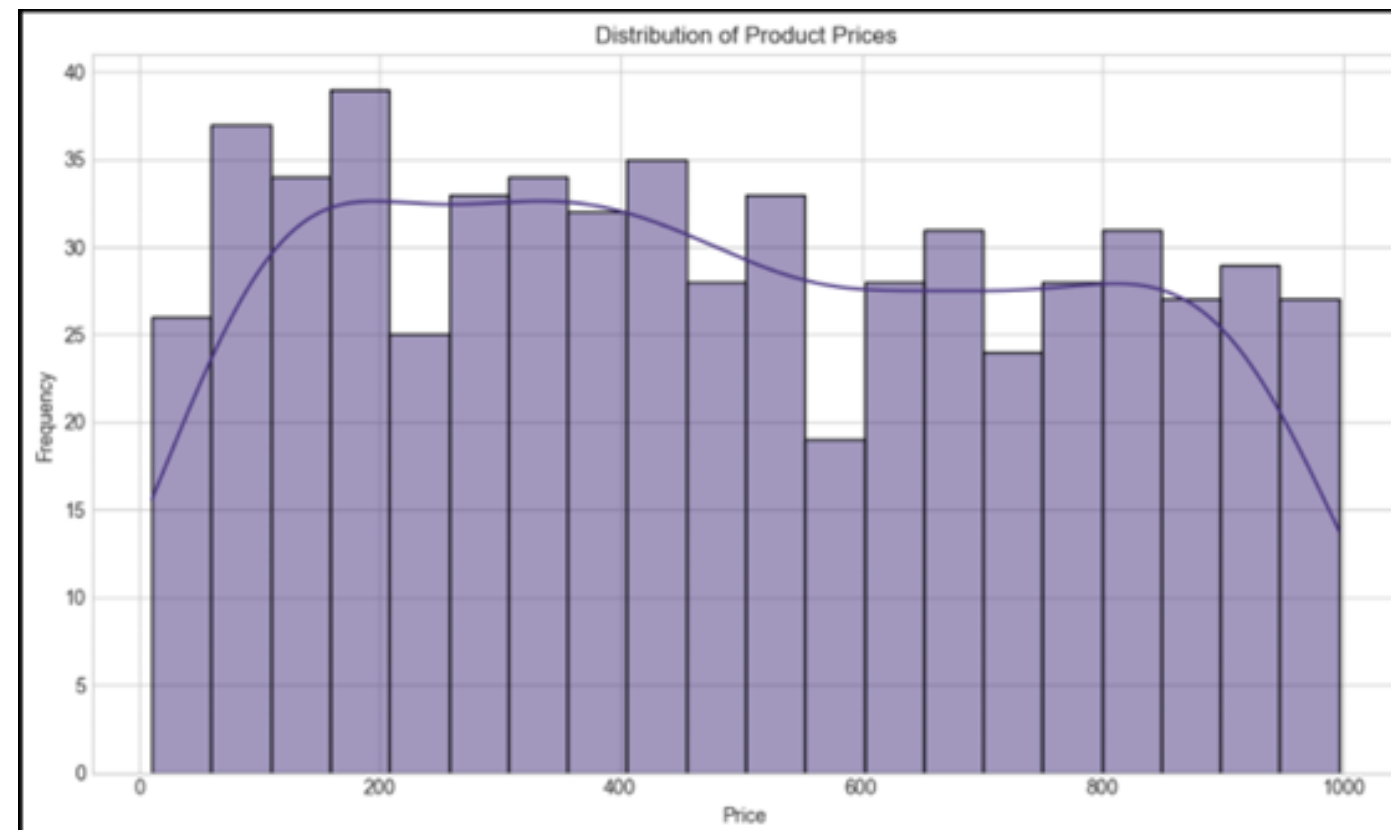
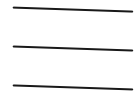


Insight: Identified high-performing products and peak sales periods. Detected imbalance in order statuses indicating potential operational bottlenecks.

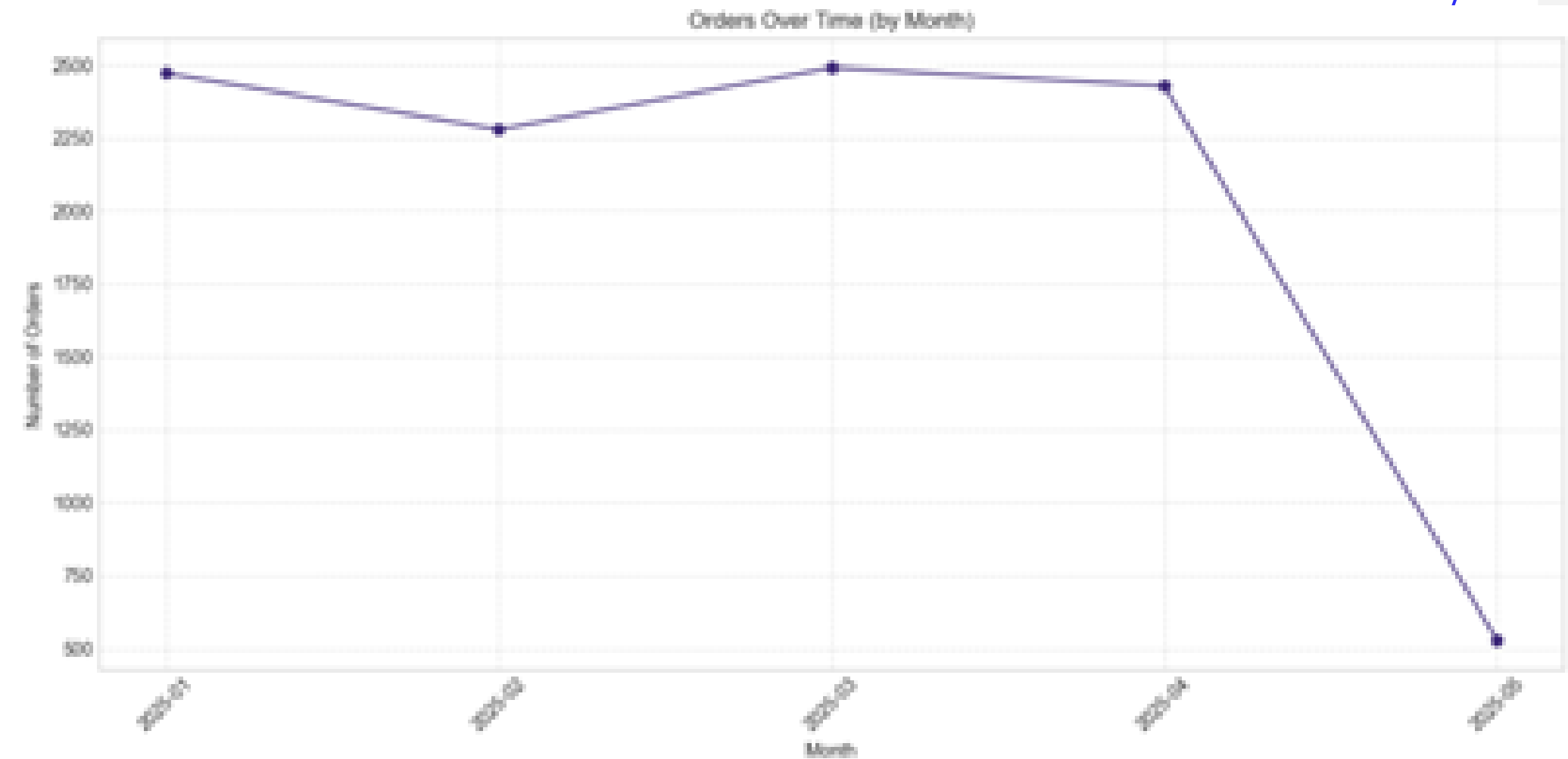
	id	name	parent_id	parent_name
0	1	Part	NaN	NaN
1	2	Program	NaN	NaN
2	3	Large	NaN	NaN
3	4	Hold	NaN	NaN
4	5	Speech	NaN	NaN
5	6	Born	3.0	Large
6	7	Recent	2.0	Program
7	8	Window	2.0	Program
8	9	Half	4.0	Hold
9	10	Adult	5.0	Speech
10	11	Table	NaN	NaN
11	12	Mission	NaN	NaN
12	13	Share	NaN	NaN
13	14	Free	NaN	NaN
14	15	Letter	NaN	NaN
15	16	Marriage	NaN	NaN
16	17	Who	NaN	NaN
17	18	Game	NaN	NaN
18	19	Choose	NaN	NaN
19	20	Energy	NaN	NaN
20	21	Over	5.0	Speech
21	22	Carry	8.0	Window
22	23	Glass	5.0	Speech
23	24	Or	9.0	Half
...				
26	27	Himself	9.0	Half
27	28	Remember	2.0	Program
28	29	Onto	4.0	Hold
29	30	Almost	8.0	Window

Output is truncated. View as a [scrollable element](#) or open in a [text editor](#). Adjust cell output [settings...](#)

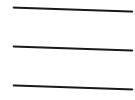




Explore product price distribution



Explore order trends over time



Feature Engineering:

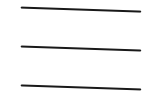
Created new variables such as:

days_as_customer: Days since a customer joined

order_month, order_day_of_week: Useful for identifying seasonality

customer_avg_order: Calculated per-customer order value trends

Repeat buyer flags and historical purchase metrics



Insight: Time-based and customer-centric features significantly improved model accuracy and personalization capabilities.

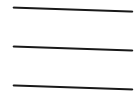
```
1 # Convert dates to datetime
2 df_merged['registration_date'] = pd.to_datetime(df_merged['registration_date'])
3 df_merged['order_date'] = pd.to_datetime(df_merged['order_date'])

1 # Calculate days as customer
2 df_merged['days_as_customer'] = (df_merged['order_date'] - df_merged['registration_date']).dt.days

1 # Create month and day of week features
2 df_merged['order_month'] = df_merged['order_date'].dt.month
3 df_merged['order_day_of_week'] = df_merged['order_date'].dt.dayofweek

1 # Calculate customer average order value (excluding current order)
2 customer_avg_order = df_merged.groupby('customer_id')['total_amount'].transform('mean')
3 df_merged['customer_avg_order'] = customer_avg_order

1 print("\nFeature engineering completed.")
2 print("New features added: days_as_customer, order_month, order_day_of_week, customer_avg_order")
3 df_merged[['days_as_customer', 'order_month', 'order_day_of_week', 'customer_avg_order']].head()
```







App Screen

Deploy 



Order Amount Predictor

This application predicts the expected order amount based on various customer and order parameters. Fill in the details below to get a prediction!

Customer Information

Days as Customer

30

- +

Customer Average Order Value (\$)

150.00

- +

Order Information

 Number of Products 

2

- +

 Total Quantity 

3

- +

Order Month 

January

▼

Order Day of Week 

Monday

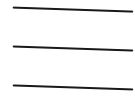
▼

Predict Order Amount

Built with  using Streamlit

Activate Windows

Go to Settings to activate Windows.



App Screen

Order Amount Predictor

This application predicts the expected order amount based on various customer and order parameters. Fill in the details below to get a prediction!

Customer Information

Days as Customer

30

- +

Customer Average Order Value (\$)

150.00

- +

Order Information

Number of Products

2

- +

Total Quantity

3

- +

Order Month

January

▼

Order Day of Week

Monday

▼

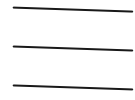
Predict Order Amount

Predicted Order Amount

\$1447.90

Activate Windows

Go to Settings to activate Windows.



Thank You For Your Time